PSTricks

pst-tree

Nodes and Trees; v.1.14

September 4, 2021

Documentation by
Herbert Voß

Package author(s):
Timothy Van Zandt
Herbert Voß

Contents

1 Overview 4

2 Tree Nodes 4

3 Tree orientation 6

4 The distance between successors 8

5 Spacing between the root and successors 9

6 Edges 9

7 Edge and node labels 11

8 Details 13

9 The scope of parameter changes 16

10 Draft mode 17

11 List of all optional arguments for pst-tree 18

References 18

The node and node connections are perfect tools for making trees, but positioning the nodes using `\rput` would be rather tedious.[1] The file `pst-tree.tex`/`pstree.sty` contains a high-level interface for making trees.

It should be noted that the correct result is not guaranteed with every `dvips` driver. This package was written for Rokicki's `dvips` programme, which is practically part of every TEX distribution.

Thanks to: Andreas Nolda

1 Unless you have a computer program that generates the coordinates.

1 Overview

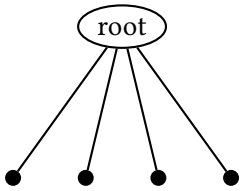
The tree commands are

```
\pstree{<root>}{<successors>}
```

TEX version	LATEX version
\psTree{<root>}	\begin{psTree}{root}
<successors>	<successors>
\endpsTree	\end{psTree}

These do the same thing, but just have different syntax. \psTree is the “long” version. These macros make a box that encloses all the nodes, and whose baseline passes through the center of the root. Most of the nodes has a variant for use within a tree and are called tree nodes (see Section 2).

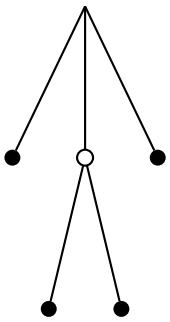
Trees and tree nodes are called *tree objects*. The *root* of a tree should be a single tree object, and the *successors* should be one or more tree objects. Here is an example with only nodes:




```
\pstree[radius=3pt]{\Toval{root}}{\TC* \TC* \TC* \TC*}
```

There is no difference between a terminal node and a root node, other than their position in the \pstree{} command.

Here is an example where a tree is included in the list of successors, and hence becomes subtree:



```
\pstree[radius=3pt]{\Tp}{%
  \TC*
  \pstree{\TC}{\TC* \TC*}
  \TC*}
```

2 Tree Nodes

In each case, the name of the tree node is formed by omitting ”‘node”’ from the end of the name and adding ”T” at the beginning. For example, \psovalnode becomes \Toval. Here is the list of such tree nodes:

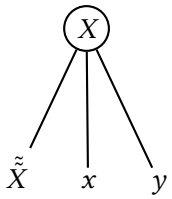
```
\Tp * [Options]
\Tc * [Options] {dim}
\TC * [Options]
\Tf * [Options]
\Tdot * [Options]
\Tr * [Options] {stuff}
\TR * [Options] {stuff}
\Tcircle * [Options] {stuff}
\TCircle * [Options] {stuff}
\Toval * [Options] {stuff}
\Tdia * [Options] {stuff}
\Ttri * [Options] {stuff}
```

The syntax of a tree node is the same as of its corresponding "normal" node, except that:

- There is always an optional argument for setting graphics parameters, even if the original node did not have one;
- There is no argument for specifying the name of the node;
- There is never a coordinate argument for positioning the node; and
- To set the reference point with `\Tr`, set the `ref` parameter.

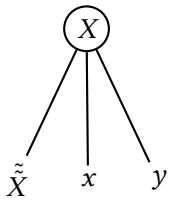
Figure 1 gives a reminder of what the nodes look like.

The difference between `\Tr` and `\TR` (variants of `\rnode` and `\Rnode`, respectively) is important with trees. Usually, you want to use `\TR` with vertical trees because the baselines of the text in the nodes line up horizontally. For example:




```
$
\pstree[nodesepB=3pt]{\Tcircle{X}}{%
 \TR{\tilde{\tilde{X}}}
 \TR{x}
 \TR{y}}
$
```

Compare with this example, which uses `\Tr`:




```
$
\pstree[nodesepB=3pt]{\Tcircle{X}}{%
 \Tr{\tilde{\tilde{X}}}
 \Tr{x}
 \Tr{y}}
$
```

There is also a null tree node:

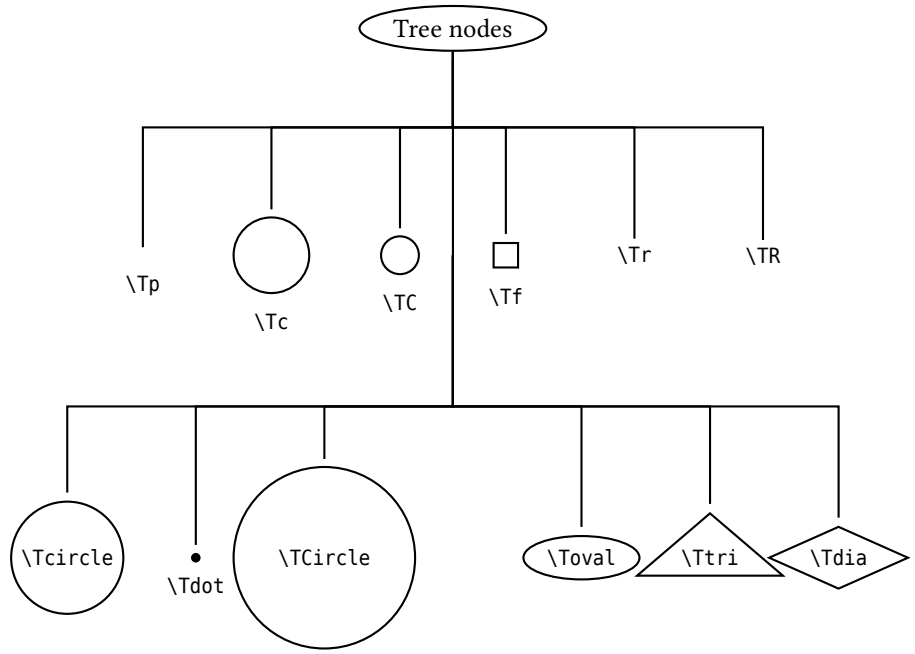
```
\Tn
```

It is meant to be just a place holder. Look at the tree in Figure page 6. The bottom row has a node missing in the middle. `\Tn{}` was used for this missing node.

There is also a special tree node that doesn't have a "normal" version and that can't be used as the root node of a whole tree:

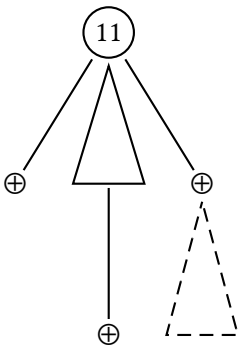
```
\Tfan * [Options]
```

This draws a triangle whose base is `fansize` and whose opposite corner is the predecessor node, adjusted by the value of `nodesepA` and `offsetA`. For example:




```
\small
\psset{armB=1cm, levelsep=3cm, treesep=-3mm, angleB=-90, angleA=90, nodesepA=3pt}
\def\s#1{#1~{\ttfamily\string#1}}\def\b#1{#1{\ttfamily\string#1}}\def\psedge#1#2{\ncangle{#2}{#1}}
\psTree[treenodesize=1cm]{\Toval{Tree nodes}}
 \s\Tp\Tc{.5}~{\ttfamily\string\Tc} \s\TC
 \psTree[levelsep=4cm,armB=2cm]{\Tp[edge=\ncline]}
  \b\Tcircle \s\Tdot
  \TCircle[radius=1.2]{\ttfamily\string\TCircle}
  \Tn \b\Toval \b\Ttri \b\Tdia
 \endpsTree
 \s\Tf \b\Tr \b\TR
\endpsTree
```

Figure 1: The tree nodes.

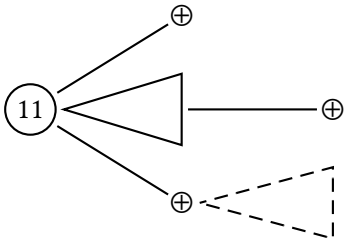



```
\pstree[dotstyle=oplus,dotsize=8pt,nodesep=2pt]{\Tcircle
    {11}}{%
  \Tdot
  \pstree{\Tfan}{\Tdot}
  \pstree{\Tdot}{\Tfan[linestyle=dashed]}}
```

3 Tree orientation

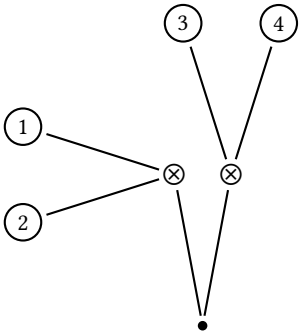
Trees can grow down, up, right or left, depending on the `treemode=` `D`, `U`, `R`, or `L` parameter.

Here is what the previous example looks like when it grows to the right:



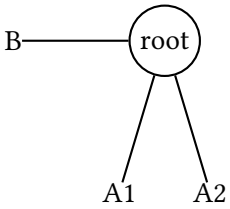

```
\pstree[dotstyle=oplus,dotsize=8pt,
 nodesep=2pt,treemode=R]
 {\Tcircle{11}}{%
  \Tdot
  \pstree{\Tfan}{\Tdot}
  \pstree{\Tdot}{\Tfan[linestyle=dashed]}}
```

You can change the `treemode` in the middle of the tree. For example, here is a tree that grows up, and that has a subtree which grows to the left:



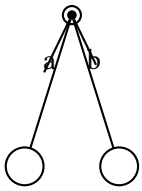

```
\footnotesize
\pstree[treemode=U,dotstyle=otimes,dotsize=8pt,nodesep=2pt]
 {\Tdot}{%
  \pstree[treemode=L]{\Tdot}{\Tcircle{1} \Tcircle{2}}
  \pstree{\Tdot}{\Tcircle{3} \Tcircle{4}}}
```

Since you can change a tree's orientation, it can make sense to include a tree (<treeB>) as a root node (of <treeA>). This makes a single logical tree, whose root is the root of <treeB>, and that has successors going off in different directions, depending on whether they appear as a successor to <treeA> or to <treeB>.



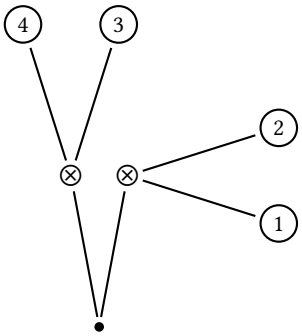

```
\pstree{\pstree[treemode=L]{\Tcircle{root}}{\Tr{B}}}{%
 \Tr{A1}
 \Tr{A2}}
```

On a semi-related theme, note that any node that creates an LR-box can contain a tree. However, nested trees of this kind are not related in any way to the rest of the tree. Here is an example:




```
\psTree{\Tcircle{%
       \pstree[treesep=0.4,levelsep=0.6,
             nodesepB=-6pt]{\Tdot}{%
        \TR{a} \TR{b}}}}
 \TC
 \TC
\endpsTree
```

When the tree grows up or down, the successors are lined up from left to right in the order they appear in `\pstree`. When the tree grows to the left or right, the successors are lined up from top to bottom. As an afterthought, you might want to flip the order of the nodes. The keyword `treeflip=true/false`let's you do this. For example:




```
\footnotesize
\pstree[treemode=U,dotstyle=otimes,dotsize=8pt,
 nodesep=2pt,treeflip=true]{\Tdot}{%
  \pstree[treemode=R]{\Tdot}{\Tcircle{1} \Tcircle{2}}
  \pstree{\Tdot}{\Tcircle{3} \Tcircle{4}}}
```

Note that I still have to go back and change the `treemode` of the subtree that used to grow to the left.

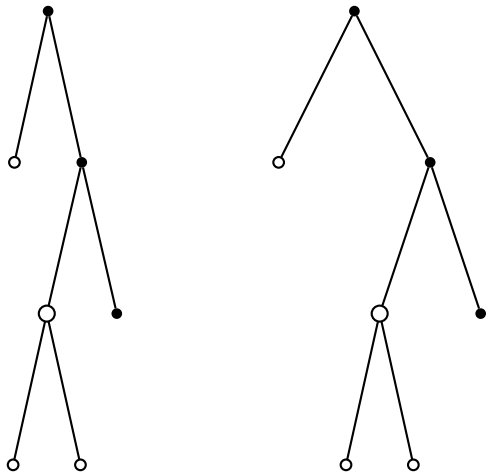
4 The distance between successors

The distance between successors is set by the key `treesep`. The rest of this section describes ways to fine-tune the spacing between successors. You can change the method for calculating the distance between subtrees by setting the `treefit=tight/loose` parameter. Here are the two methods:

`tight` When `treefit=tight` , which is the default, `treesep` is the minimum distance between each of the levels of the subtrees.

`loose` When `treefit=loose` , `treesep` is the distance between the subtrees' bounding boxes. Except when you have large intermediate nodes, the effect is that the horizontal distance (or vertical distance, for horizontal trees) between all the terminal nodes is the same (even when they are on different levels).[2]

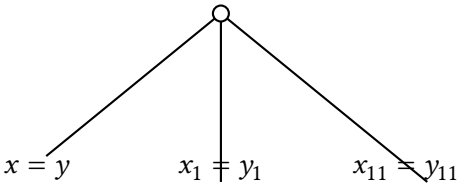
Compare:



With `treefit=loose` , trees take up more space, but sometimes the structure of the tree is emphasized.

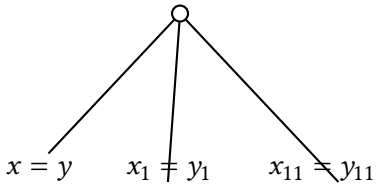
Sometimes you want the spacing between the centers of the nodes to be regular even though the nodes have different sizes. If you set `treenodesize` to a non-negative value, then PSTricks sets the width (or height+depth for vertical trees) to `treenodesize`, *for the purpose of calculating the distance between successors.*

For example, ternary trees look nice when they are symmetric, as in the following example:




```
\pstree[nodesepB=-8pt,treenodesize=.85]{\Tc{3pt}}{%
  \TR{$x=y$}
  \TR{$x_1=y_1$}
  \TR{$x_{11}=y_{11}$}}%$
```

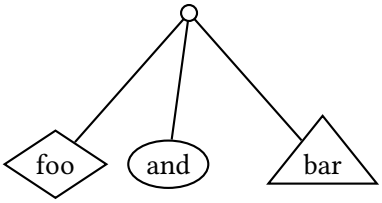
Compare with this example, where the spacing varies with the size of the nodes:




```
\pstree[nodesepB=-8pt]{\Tc{3pt}}{%
  \TR{$x=y$}
  \TR{$x_1=y_1$}
  \TR{$x_{11}=y_{11}$}}%$
```

Finally, if all else fails, you can adjust the distance between two successors by inserting `\tspace{`*length*`}` between them:

2 When all the terminal nodes are on the same level, and the intermediate nodes are not wider than the base of their corresponding subtrees, then there is no difference between the two methods.



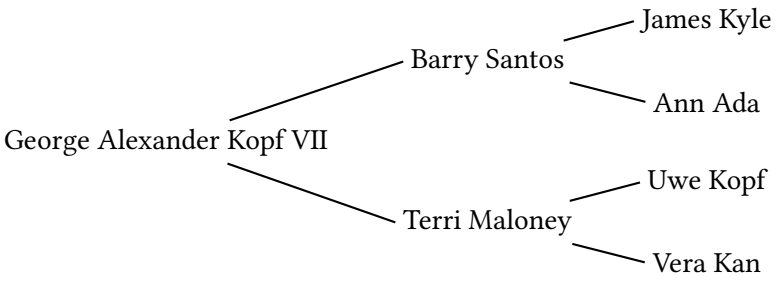

```
\pstree{\Tc{3pt}}{%
 \Tdia{foo}
 \tspace{-0.5}
 \Toval{and}
 \Ttri{bar}}
```

5 Spacing between the root and successors

The distance between the center lines of the tree levels is `levelsep`. If you want the spacing between levels to vary with the size of the levels, use the * convention. Then `levelsep` is the distance between the bottom of one level and the top of the next level (or between the sides of the two levels, for horizontal trees).

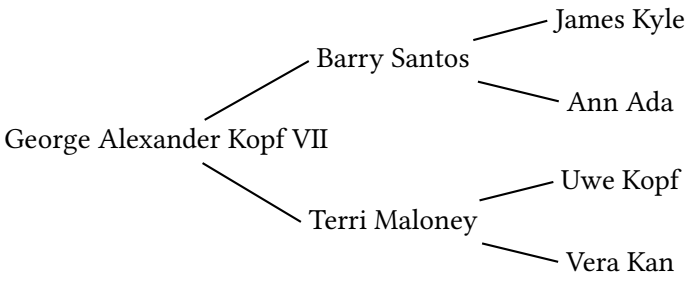
Note: PSTricks has to write some information to your `.aux` file if using LaTeX, or to `\jobname.pst` otherwise, in order to calculate the spacing. You have to run your input file a few times before PSTricks gets the spacing right.

trees. Compare the following example:




```
\def\psedge#1#2{\ncdiagg[nodesep=3pt,angleA=180,armA=0]{#2}{#1}}
\pstree[treemode=R,levelsep=*1cm]{\Tr{George Alexander Kopf VII}}{%
   \pstree{\Tr{Barry Santos}}{\Tr{James Kyle} \Tr{Ann Ada}}
   \pstree{\Tr{Terri Maloney}}{\Tr{Uwe Kopf} \Tr{Vera Kan}}}
```

with this one, were the spacing between levels is fixed:




```
\def\psedge#1#2{\ncdiagg[nodesep=3pt,angleA=180,armA=0]{#2}{#1}}
\pstree[treemode=R,levelsep=3cm]{\Tr{George Alexander Kopf VII}}{%
   \pstree{\Tr{Barry Santos}}{\Tr{James Kyle} \Tr{Ann Ada}}
   \pstree{\Tr{Terri Maloney}}{\Tr{Uwe Kopf} \Tr{Vera Kan}}}
```

6 Edges

Right after you use a tree node command, `\pssucc` is equal to the name of the node, and `\pspred` is equal to the name of the node's predecessor. Therefore, you can draw a line between the node and its predecessor by inserting, for example,

```
\ncline{\pspred}{\pssucc}
```

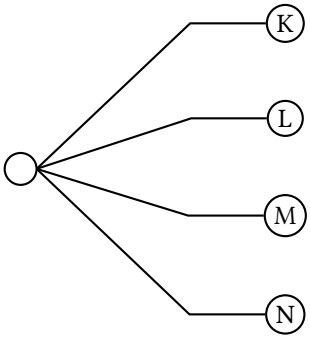
To save you the trouble of doing this for every node, each tree node executes

```
\psedge{\pspred}{\pssucc}
```

The default definition of \psedge is \ncline, but you can redefine it as you please with \def or LaTeX's \renewcommand.

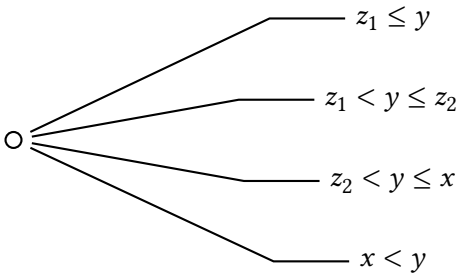
For example, here I use \ncdiag, with armA=0, to get all the node connections to emanate from the same point in the predecessor. LaTeX users can instead type:

```
\renewcommand{\psedge}{\ncdiag[armA=0,angleB=180,armB=1cm]}
```



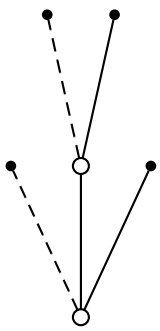

```
\def\psedge{\ncdiag[armA=0,angleB=180,armB=1cm]}
\pstree[treemode=R,levelsep=3.5cm,framesep=2pt]{\Tc{6pt}}{%
   \small \Tcircle{K} \Tcircle{L} \Tcircle{M} \Tcircle{N}}
```

Here is an example with \ncdiagg. Note the use of a negative armA value so that the corners of the edges are vertically aligned, even though the nodes have different sizes:




```
$
\def\psedge#1#2{\ncdiagg[angleA=180,armA=1cm,nodesep=4pt]{#2}{#1}}
% Or: \renewcommand{\psedge}[2]{ ... }
\pstree[treemode=R, levelsep=5cm]{\Tc{3pt}}{%
  \Tr{z_1\leq y} \Tr{z_1<y\leq z_2} \Tr{z_2<y\leq x} \Tr{x<y}
}
$
```

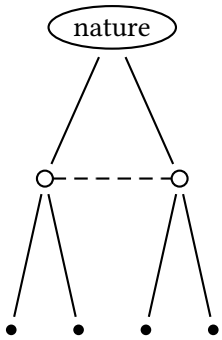
Another way to define \psedge{} is with the edge parameter. Be sure to enclose the value in braces "" if it contains commas or other parameter delimiters. This gets messy if your command is long, and you can't use arguments like in the preceding example, but for simple changes it is useful. For example, if I want to switch between a few node connections frequently, I might define a command for each node connection, and then use the edge parameter.



```
\def\dedge{\ncline[linestyle=dashed]}
\pstree[treemode=U,radius=2pt]{\Tc{3pt}}{%
  \TC*[edge=\dedge]
  \pstree{\Tc{3pt}}{\TC*[edge=\dedge] \TC*}
  \TC*}
```

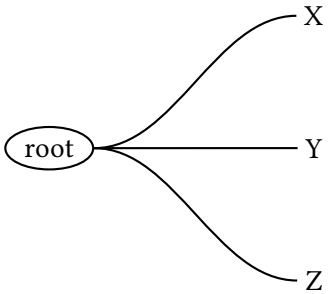
You can also set edge=none to suppress the node connection.

If you want to draw a node connection between two nodes that are not direct predecessor and successor, you have to give the nodes a name that you can refer to, using the name parameter. For example, here I connect two nodes on the same level:

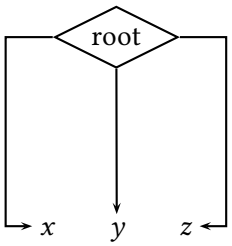



```
\pstree[nodesep=3pt,radius=2pt]{\Toval{nature}}{%
 \pstree{\Tc[name=top]{3pt}}{\TC* \TC*}
 \pstree{\Tc[name=bot]{3pt}}{\TC* \TC*}}
\ncline[linestyle=dashed]{top}{bot}
```

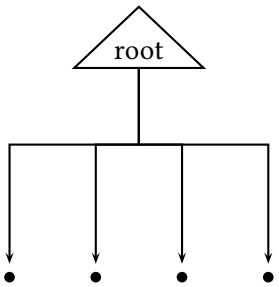
We conclude with the more examples.




```
\def\psedge{\nccurve[angleB=180, nodesepB=3pt]}
\pstree[treemode=R, treesep=1.5, levelsep=3.5]%
 {\Toval{root}}{\Tr{X} \Tr{Y} \Tr{Z}}
```



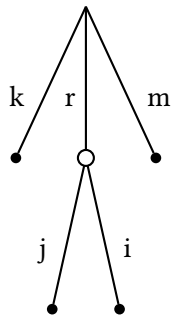

```
\pstree[nodesepB=3pt, arrows=->, xbbl=15pt,
 xbbr=15pt, levelsep=2.5cm]{\Tdia{root}}{%
 $
 \TR[edge={\ncbar[angle=180]}]{x}
 \TR{y}
 \TR[edge=\ncbar]{z}
 $}
```




```
\psset{armB=1cm, levelsep=3cm, treesep=1cm,
 angleB=-90, angleA=90, arrows=<-, nodesepA=3pt}
\def\psedge#1#2{\ncangle{#2}{#1}}
\pstree[radius=2pt]{\Ttri{root}}{\TC* \TC* \TC* \TC*}
```

7 Edge and node labels

Right after a node, an edge has typically been drawn, and you can attach labels using `\ncput`, `\tlput`, etc. With `\tlput`, `\trput`, `\taput`, and `\tbput`, you can align the labels vertically or horizontally, just like the nodes. This can look nice, at least if the slopes of the node connections are not too different.



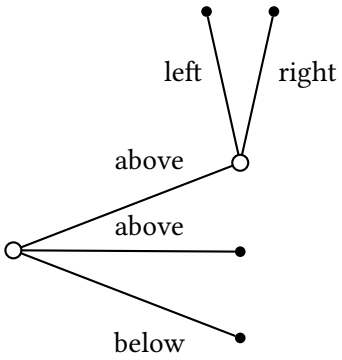

```
\pstree[radius=2pt]{\Tp}{%
 \psset{tpos=.6}
 \TC* \tlput{k}
 \pstree{\Tc{3pt} \tlput[labelsep=3pt]{r}}{%
  \TC* \tlput{j}
  \TC* \trput{i}}
 \TC* \trput{m}}
```

Within trees, the tpos parameter measures this distance from the predecessor to the successor, whatever the orientation of the true. (Outside of trees it measures the distance from the top to bottom or left to right nodes.)

PSTricks also sets shortput=tab within trees. This is a special shortput option that should not be used outside of trees. It implements the following abbreviations, which depend of the orientation of the true:

	Short for:	
Char.	*Vert.*	*Horiz.*
^	\tlput	\taput
_	\trput	\tbput

(The scheme is reversed if treeflip=true .)

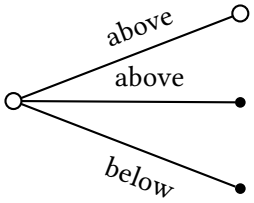



```
\psset{tpos=.6}
\pstree[treemode=R, thistreesep=1cm,
 thislevelsep=3cm,radius=2pt]{\Tc{3pt}}{%
 \pstree[treemode=U, xbbr=20pt]{\Tc{3pt}^{above}}{%
  \TC*^{left}
  \TC*_{right}}
 \TC*^{above}
 \TC*_{below}}
```

You can change the character abbreviations with

```
\MakeShortTab{<char1>}{<char2>}
```

The \n*put commands can also give good results:




```
\psset{npos=.6,nrot=:U}
\pstree[treemode=R, thistreesep=1cm,
 thislevelsep=3cm]{\Tc{3pt}}{%
 \Tc{3pt}\naput{above}
 \Tc*{2pt}\naput{above}
 \Tc*{2pt}\nbput{below}}
```

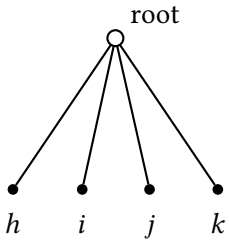
You can put labels on the nodes using \nput. However, \pstree won't take these labels into account when calculating the bounding boxes.

There is a special node label option for trees that does keep track of the bounding boxes:

```
* [Options] {stuff}
```

Call this a "tree node label".

Put a tree node label right after the node to which it applies, before any node connection labels (but node connection labels, including the short forms, can follow a tree node label). The label is positioned directly below the node in vertical trees, and similarly in other trees. For example:




```
\pstree[radius=2pt]{\Tc{3pt}\nput{45}{\pssucc}{root}}{%
 \TC*~{$h$} \TC*~{$i$} \TC*~{$j$} \TC*~{$k$}}
```

Note that there is no “long form” for this tree node label. However, you can change the single character used to delimit the label with

```
\MakeShortTnput{<char1>}
```

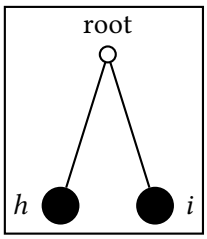
If you find it confusing to use a single character, you can also use a command sequence. E.g.,

```
\MakeShortTnput{\tnput}
```

You can have multiple labels, but each successive label is positioned relative to the bounding box that includes the previous labels. Thus, the order in which the labels are placed makes a difference, and not all combinations will produce satisfactory results.

You will probably find that the tree node label works well for terminal nodes, without your intervention. However, you can control the tree node labels be setting several parameters.

To position the label on any side of the node (”l”eft, ”r”ight, ”a”bove or ”b”elow), set: `tnpos=l/r/a/b`




```
\psframebox{%
  \pstree{\Tc{3pt}~[tnpos=a,tndepth=0pt,radius=4pt]{root}}{%
    \TC*~[tnpos=l]{$h$}
    \TC*~[tnpos=r]{$i$}}}
```

When you leave the argument empty, which is the default, PSTricks chooses the label position is automatically.

To change the distance between the node and the label, set `tnsep` to a dimension When you leave the argument empty, which is the default, PSTricks uses the value of `labelsep`. When the value is negative, the distance is measured from the center of the node.

When labels are positioned below a node, the label is given a minimum height of `tnheight`. Thus, if you add labels to several nodes that are horizontally aligned, and if either these nodes have the same depth or `tnsep` is negative, and if the height of each of the labels is no more than `tnheight`, then the labels will also be aligned by their baselines. The default is `\ht\strutbox`, which in most TEX formats is the height of a typical line of text in the current font. Note that the value of `tnheight` is not evaluated until it is used.

The positioning is similar for labels that go below a node. The label is given a minimum *depth* of `tndepth`. For labels positioned above or below, the horizontal reference point of the label, i.e., the point in the label directly above or below the center of the node, is set by the `href` parameter.

When labels are positioned on the left or right, the right or left edge of the label is positioned distance `tnsep` from the node. The vertical point that is aligned with the center of the node is set by `tnyref`. When you leave this empty, `vref` is used instead. Recall that `vref` gives the vertical *distance* from the baseline. Otherwise, the `tnyref` parameter works like the `yref` parameter, giving the fraction of the distance from the bottom to the top of the label.

8 Details

PSTricks does a pretty good job of positioning the nodes and creating a box whose size is close to the true bounding box of the tree. However, PSTricks does not take into account the node connections or labels when calculating the bounding boxes, except the tree node labels.

If, for this or other reasons, you want to fine tune the bounding box of the nodes, you can set the following parameters to a dimension:

name	*default*
bbl	0pt
bbr	0pt
bbh	0pt
bbd	0pt
xbbl	0pt
xbbr	0pt
xbbh	0pt
xbbd	0pt

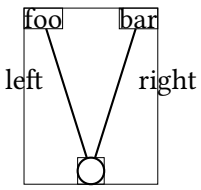
The "'x"' versions increase the bounding box by <dim>, and the others set the bounding box to the dimension. There is one parameter for each direction from the center of the node, **l**eft, **r**ight, **h**eight, and **d**epth.

These parameters affect trees and nodes, and subtrees that switch directions, but not subtrees that go in the same direction as their parent tree (such subtrees have a profile rather than a bounding box, and should be adjusted by changing the bounding boxes of the constituent nodes).

Save any fiddling with the bounding box until you are otherwise finished with the tree.

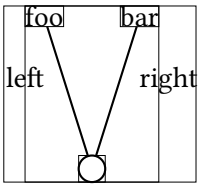
You can see the bounding boxes by setting the `showbbox=true/false`parameter to `true`. To see the bounding boxes of all the nodes in a tree, you have to set this parameter before the tree.

In the following example, the labels stick out of the bounding box:



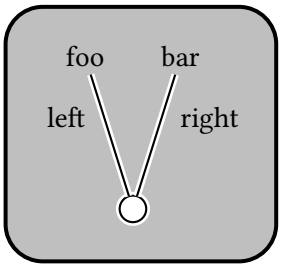

```
\psset{tpos=.6,showbbox=true}
\pstree[treemode=U]{\Tc{5pt}}{%
   \TR{foo}^{left}
   \TR{bar}_{right}}
```

Here is how we fix it:




```
\psset{tpos=.6,showbbox=true}
\pstree[treemode=U,xbbl=8pt,xbbr=14pt]{\Tc{5pt}}{%
   \TR{foo}^{left}
   \TR{bar}_{right}}
```

Now we can frame the tree:



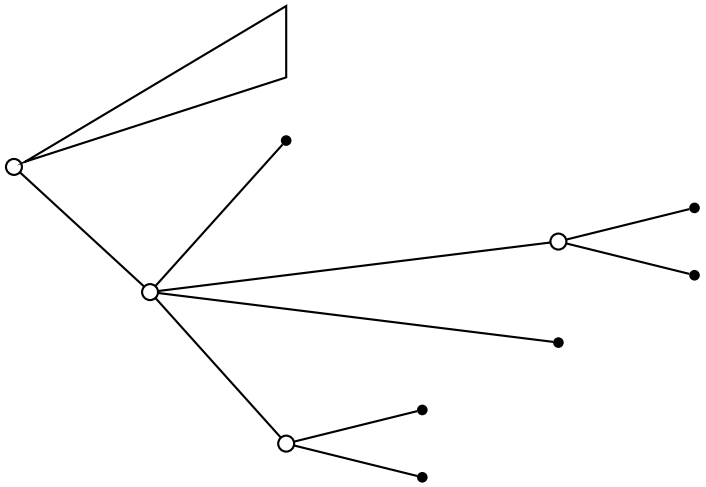

```
\psframebox[fillstyle=solid,fillcolor=lightgray,framesep=14pt,
 linearc=14pt,cornersize=absolute,linewidth=1.5pt]{%
 \psset{tpos=.6,border=1pt,nodesepB=3pt}
 \pstree[treemode=U,xbbl=8pt,xbbr=14pt]{%
  \Tc[fillcolor=white,fillstyle=solid]{5pt}}{%
  \TR*{foo}^{left}
  \TR*{bar}_{right}}}
```

We would have gotten the same result by changing the bounding box of the two terminal nodes.

To skip levels, use

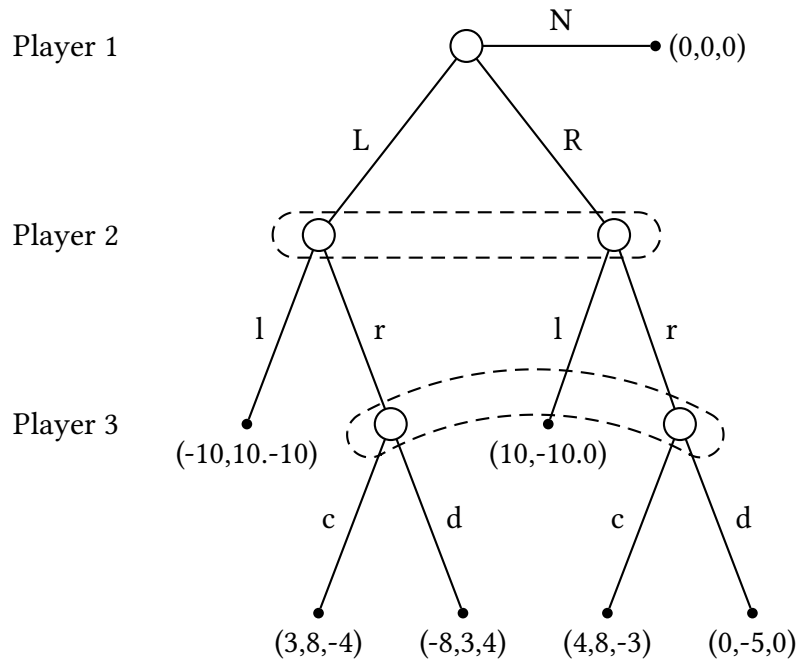
```
\skiplevel * [Options] {nodes or subtrees}
\skiplevels * [Options] {int}
     <nodes or subtrees>
\endskiplevels
```

These are kind of like subtrees, but with no root node.



```
\pstree[treemode=R,levelsep=1.8,radius=2pt]{\Tc{3pt}}{%
 \skiplevel{\Tfan}
 \pstree{\Tc{3pt}}{%
  \TC*
  \skiplevels{2}
   \pstree{\Tc{3pt}}{\TC* \TC*}
   \TC*
  \endskiplevels
  \pstree{\Tc{3pt}}{\TC* \TC*}}}
```

The profile at the missing levels is the same as at the first non-missing level. You can adjust this with the bounding box parameters. You get greatest control if you use nested `\skiplevel` commands instead of `\skiplevels`.



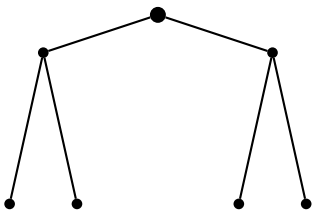

```
\large\psset{radius=6pt, dotsize=4pt}
\pstree[thislevelsep=0,edge=none,levelsep=2.5cm]{\Tn}{%
   \pstree{\TR{Player 1}}{\pstree{\TR{Player 2}}{\TR{Player 3}}}
   \psset{edge=\ncline}
   \pstree{\pstree[treemode=R]{\TC}{\Tdot ~{(0,0,0)} ^{N}}}{%
       \pstree{\TC[name=A] ^{L}}{%
        \Tdot ~{(-10,10.-10)} ^{l}
        \pstree{\TC[name=C] _{r}}{%
          \Tdot ~{(3,8,-4)} ^{c}
          \Tdot ~{(-8,3,4)} _{d}}}
       \pstree{\TC[name=B] _{R}}{%
        \Tdot ~{(10,-10.0)} ^{l}
        \pstree{\TC[name=D]_{r}}{%
          \Tdot ~{(4,8,-3)} ^{c}
          \Tdot ~{(0,-5,0)} _{d}}}}}
\ncbox[linearc=.3,boxsize=.3,linestyle=dashed,nodesep=.4]{A}{B}
\ncarcbox[linearc=.3,boxsize=.3,linestyle=dashed,arcangle=25,nodesep=.4]{D}{C}
```

9 The scope of parameter changes

`edge` is the only parameter which, when set in a tree node's parameter argument, affects the drawing of the node connection (e.g., if you want to change the `nodesep`, your edge has to include the parameter change, or you have to set it before the node).

As noted at the beginning of this section, parameter changes made with `\pstree` affect all subtrees. However, there are variants of some of these parameters for making local changes, i.e, changes that affects only the current level: `thistreesep`, `thistreenodesize`, `thistreefit=tight/loose`, and `thislevelsep`.

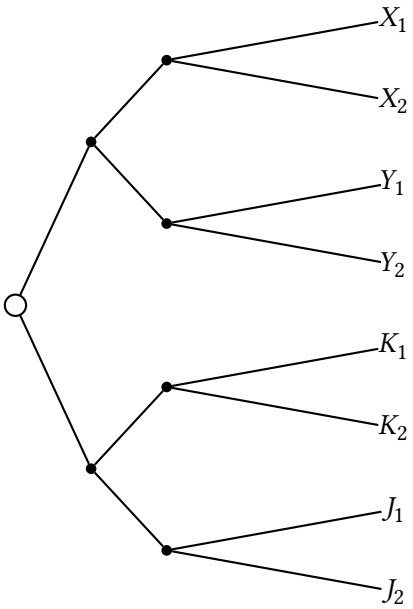
For example:



```
\pstree[thislevelsep=.5cm,thistreesep=2cm,
 radius=2pt]{\Tc*{3pt}}{%
 \pstree{\TC*}{\TC* \TC*}
 \pstree{\TC*}{\TC* \TC*}}
```

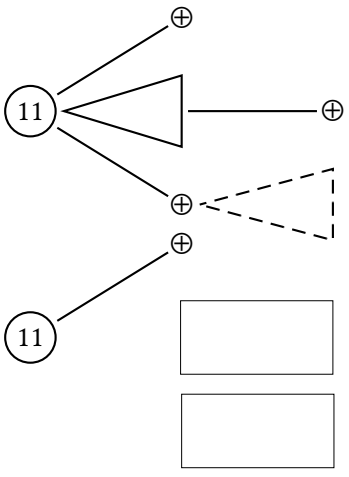
There are some things you may want set uniformly across a level in the tree, such as the `levelsep`. At level `<n>`, the command `\pstreehook<roman(n)>` (e. g., `\pstreehookii`) is executed, if it is defined (the root node of the

whole tree is level 0, the successor tree objects and the node connections from the root node to these successors is level 1, etc.). In the following example, the `levelsep` is changed for level 2, without having to set the `thislevelsep` parameter for each of the three subtrees that make of level 2:




```
\[
\def\pstreehookiii{\psset{thislevelsep=3cm}}
\pstree[treemode=R,levelsep=1cm,radius=2pt]{\Tc{4pt}}{%
  \pstree{\TC*}{%
    \pstree{\TC*}{\Tr{X_1} \Tr{X_2}}
    \pstree{\TC*}{\Tr{Y_1} \Tr{Y_2}}}
  \pstree{\TC*}{%
    \pstree{\TC*}{\Tr{K_1} \Tr{K_2}}
    \pstree{\TC*}{\Tr{J_1} \Tr{J_2}}}}
\]
```

10 Draft mode




```
  \pstree[dotstyle=oplus,dotsize=8pt,
    nodesep=2pt,treemode=R]
    {\Tcircle{11}}{%
      \Tdot
      \pstree{\Tfan}{\Tdot}
      \pstree{\Tdot}{\Tfan[linestyle=dashed]}}
%
  \pstree[draft,dotstyle=oplus,dotsize=8pt,
    nodesep=2pt,treemode=R]
    {\Tcircle{11}}{%
      \Tdot
      \pstree{\Tfan}{\Tdot}
      \pstree{\Tdot}{\Tfan[linestyle=dashed]}}
```

11 List of all optional arguments for pst-tree

The default value ist set when an optional argument is called without any value, e. g. `\pstree[`*levelsep*`]` is the same as `\pstree[`*levelsep=2cm*`]`. All boolean arguments are preset to `false`.

Key	Type	Default
treefit	ordinary	tight
thistreefit	ordinary	tight
treemode	ordinary	0
treesep	ordinary	0.75cm
thistreesep	ordinary	[none]
treenodesize	ordinary	-1pt
thistreenodesize	ordinary	-1pt
levelsep	ordinary	2cm
thislevelsep	ordinary	[none]
treeflip	boolean	true
showbbox	boolean	true
edge	ordinary	\ncline
bbl	ordinary	[none]
bbr	ordinary	[none]
bbh	ordinary	[none]
bbd	ordinary	[none]
xbbl	ordinary	[none]
xbbr	ordinary	[none]
xbbh	ordinary	[none]
xbbd	ordinary	[none]
fansize	ordinary	1cm
tnsep	ordinary	
tnyref	ordinary	
tnheight	ordinary	\ht \strutbox
tndepth	ordinary	\dp \strutbox
tnpos	ordinary	

References

[1] Denis Girou. “Présentation de PSTricks”. In: *Cahier GUTenberg* 16 (Apr. 1994), pp. 21–70.

[2] Michel Goosens et al. *The LaTeX Graphics Companion*. second. Boston, Mass.: Addison-Wesley Publishing Company, 2007.

[3] Nikolai G. Kollock. *PostScript richtig eingesetzt: vom Konzept zum praktischen Einsatz*. Vaterstetten: IWT, 1989.

[4] Herbert Voß. *PSTricks – Grafik für TeX und LaTeX*. 7th ed. Heidelberg/Hamburg: DANTE – Lehmanns, 2016.

[5] Timothy Van Zandt. *multido.tex - a loop macro, that supports fixed-point addition*. CTAN:/macros/generic/multido.tex, 1997.

[6] Timothy Van Zandt and Denis Girou. “Inside PSTricks”. In: *TUGboat* 15 (Sept. 1994), pp. 239–246.

Index

Symbols

□, 12

A

a, 13
armA, 10
.aux, 9

B

b, 13
bbd, 14
bbh, 14
bbl, 14
bbr, 14

D

D, 6
\def, 10
dvips, 3

E

edge, 10, 16
\endpsTree, 4
\endskiplevels, 14
Environment
- psTree, 4

Extension
- .aux, 9

F

fansize, 5

H

href, 13

J

\jobname, 9

K

Keyvalue
- D, 6
- L, 6
- R, 6
- U, 6
- a, 13
- b, 13
- l, 13
- loose, 8, 16
- r, 13
- tight, 8, 16

Keyword
- armA, 10
- bbd, 14
- bbh, 14
- bbl, 14
- bbr, 14
- edge, 10, 16
- fansize, 5
- href, 13
- labelsep, 13
- levelsep, 9, 16f
- name, 10
- nodesep, 16
- nodesepA, 5
- offsetA, 5
- ref, 5
- shortput, 12
- showbbox, 14
- thislevelsep, 16f
- thistreefit, 16
- thistreenodesize, 16
- thistreesep, 16
- tndepth, 13
- tnheight, 13
- tnpos, 13
- tnsep, 13
- tnyref, 13
- tpos, 12
- treefit, 8
- treeflip, 7, 12
- treemode, 7
- treemode=, 6
- treenodesize, 8
- treesep, 8
- vref, 13
- xbbd, 14
- xbbh, 14
- xbbl, 14
- xbbr, 14
- yref, 13

L

L, 6
l, 13
labelsep, 13
levelsep, 9, 16f
loose, 8, 16

M
Macro
- \MakeShortTab, 12
- \MakeShortTnput, 13
- \Rnode, 5
- \TC*, 5
- \TCircle*, 5
- \TR, 5
- \TR*, 5
- \Tc*, 5
- \Tcircle*, 5
- \Tdia*, 5
- \Tdot*, 5
- \Tf*, 5
- \Tfan*, 5
- \Tn, 5
- \Toval, 4
- \Toval*, 5
- \Tp*, 5
- \Tr*, 5
- \Tr, 5
- \Ttri*, 5
- \def, 10
- \endpsTree, 4
- \endskiplevels, 14
- \jobname, 9
- \ncdiag, 10
- \ncdiagg, 10
- \ncline, 10
- \ncput, 11
- \nput, 12
- \psTree, 4
- \psedge, 10
- \psovalnode, 4
- \pspred, 9
- \pssucc, 9
- \pstree, 4, 7, 12, 16, 18
- \pstreehookii, 16
- \renewcommand, 10
- \rnode, 5
- \rput, 3
- \skiplevel*, 14
- \skiplevel, 15
- \skiplevels, 15
- \skiplevels*, 14
- \strutbox, 13
- \taput, 11f
- \tbput, 11f
- \tlput, 11f
- \trput, 11f
- \tspace, 8

\MakeShortTab, 12
\MakeShortTnput, 13

N
name, 10
\ncdiag, 10
\ncdiagg, 10
\ncline, 10
\ncput, 11
nodesep, 16
nodesepA, 5
none, 10
\nput, 12

O
offsetA, 5

P
Program
- dvips, 3

psTree, 4
\psTree, 4
\psedge, 10
\psovalnode, 4
\pspred, 9
\pssucc, 9
\pstree, 4, 7, 12, 16, 18
\pstreehookii, 16

R
R, 6
r, 13
ref, 5
\renewcommand, 10
\rnode, 5
\Rnode, 5
Rokicki, 3
\rput, 3

S
shortput, 12
showbbox, 14
\skiplevel, 15
\skiplevel*, 14
\skiplevels, 15
\skiplevels*, 14
\strutbox, 13
subtree, 4, 8
Syntax
- □, 12

T

\TC*, 5
\TCircle*, 5
\TR, 5
\TR*, 5
tab, 12
\taput, 11f
\tbput, 11f
\Tc*, 5
\Tcircle*, 5
\Tdia*, 5
\Tdot*, 5
\Tf*, 5
\Tfan*, 5
thislevelsep, 16f
thistreefit, 16
thistreenodesize, 16
thistreesep, 16
tight, 8, 16
\tlput, 11f
\Tn, 5
tndepth, 13
tnheight, 13
tnpos, 13
tnsep, 13
tnyref, 13
\Toval, 4
\Toval*, 5
\Tp*, 5
tpos, 12
\Tr, 5
\Tr*, 5
tree objects, 4
treefit, 8
treeflip, 7, 12
treemode, 7
treemode=, 6
treenodesize, 8
treesep, 8
\trput, 11f
true, 12
\tspace, 8
\Ttri*, 5

U

U, 6

V

Value
- loose, 8
- none, 10
- tab, 12
- tight, 8
- true, 12

vref, 13

X

xbbd, 14
xbbh, 14
xbbl, 14
xbbr, 14

Y

yref, 13